

Assignment Title: SQL Sales Data Analysis Using MySQL Workbench

Name: Rohit Balasaheb Kokare

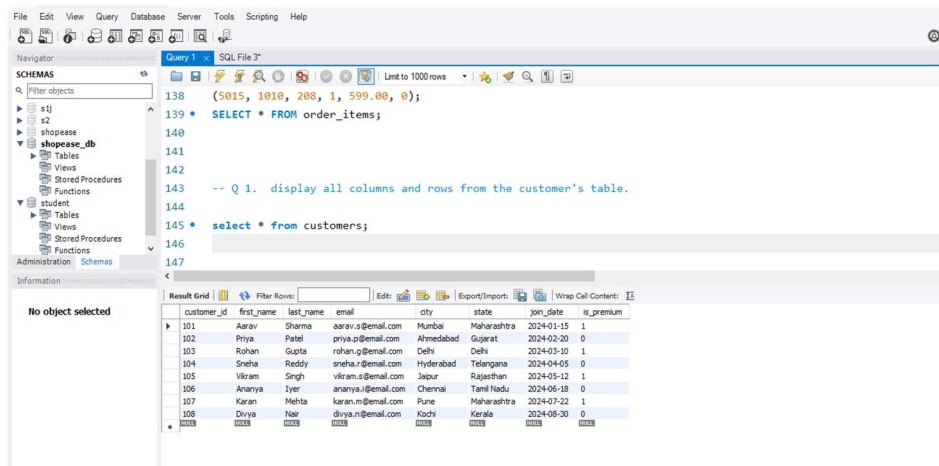
Class: B.Tech (Information Technology)

Roll No.: 83

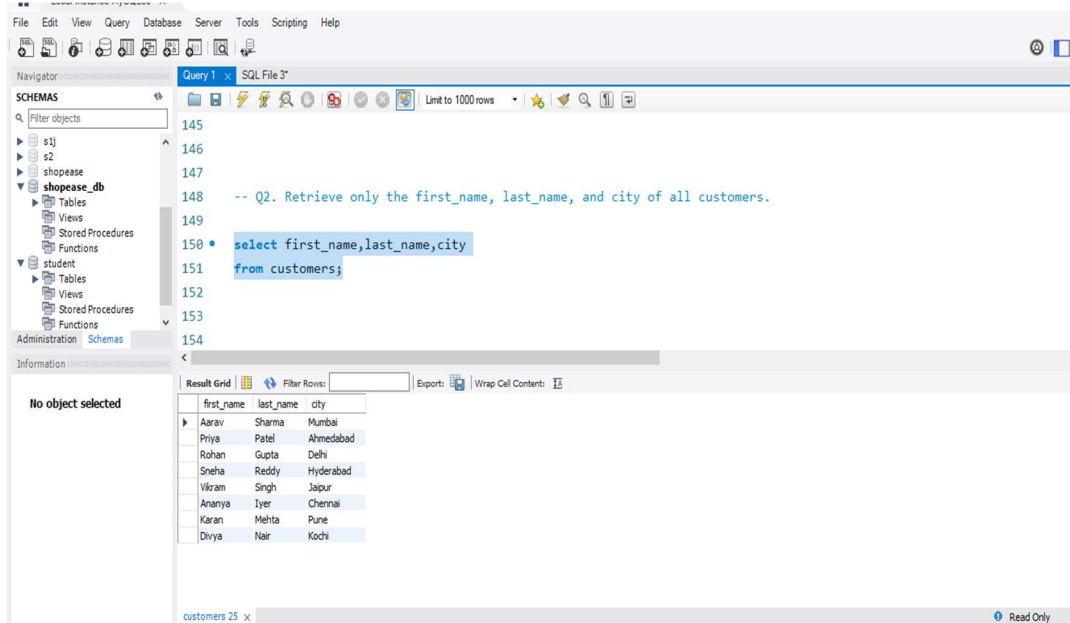
PRN No.:UIT23M1083

Section A — SQL Basics

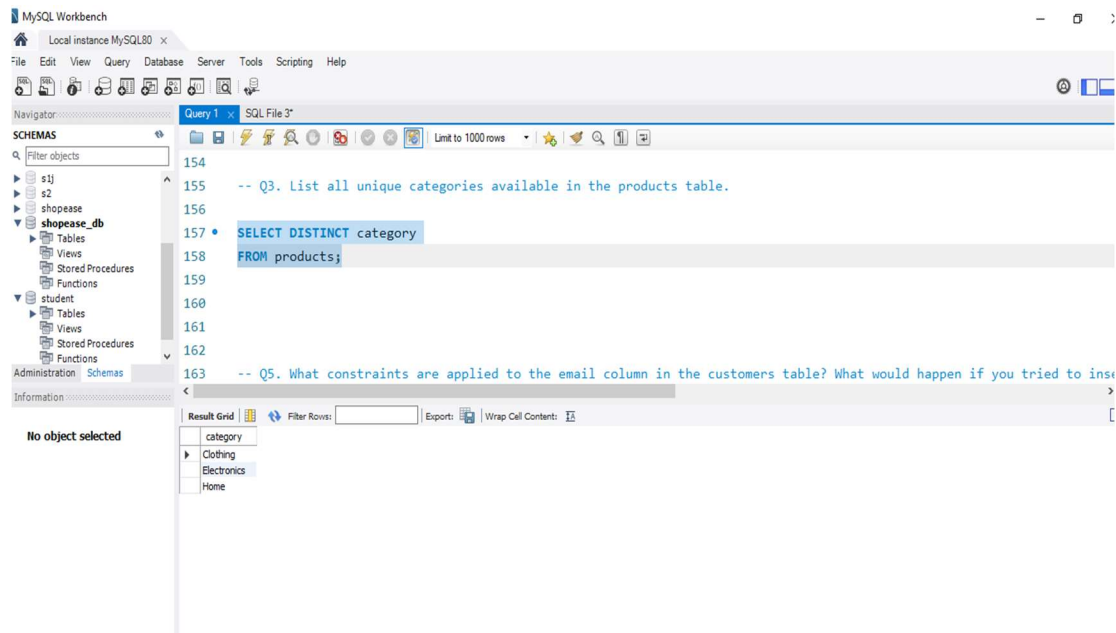
Q1. Write a query to display all columns and rows from the customer's table.



Q2. Retrieve only the first_name, last_name, and city of all customers.



Q3. List all unique categories available in the products table.



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Primary Keys:

Table	Primary Key
customers	customer_id
products	product_id
orders	order_id
order_items	item_id

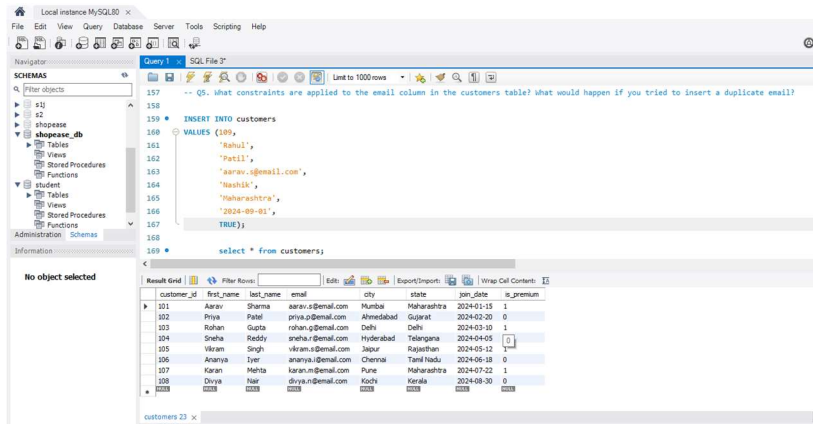
Explanation:

A **Primary Key** is a column (or set of columns) that uniquely identifies each record in a table.

A Primary Key must be:

- **Unique:** No two rows can have the same primary key value.
- **NOT NULL:** Every row must have a value for the primary key.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

-- Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

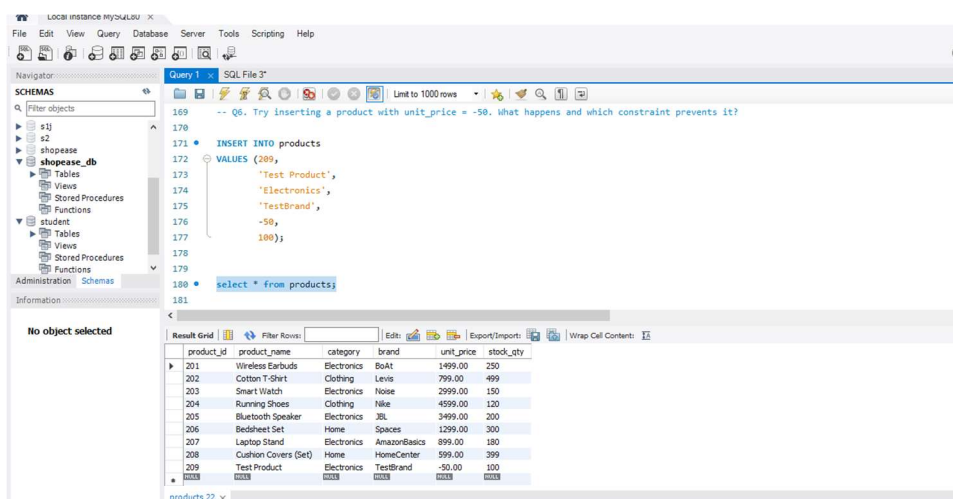
INSERT INTO customers
VALUES (400,
       'Rahul',
       'Patil',
       'aarav.s@gmail.com',
       'Nashik',
       'Maharashtra',
       '2024-09-01',
       TRUE);

select * from customers;
  
```

The result grid shows the following data for the customers table:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@gmail.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@gmail.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@gmail.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@gmail.com	Hyderabad	Telangana	2024-04-05	1
105	Vikram	Singh	vikram.s@gmail.com	Japur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@gmail.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehra	karan.m@gmail.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@gmail.com	Kochi	Kerala	2024-08-30	0

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

-- Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it?

INSERT INTO products
VALUES (209,
       'Test Product',
       'Electronics',
       'TestBrand',
       -50,
       100);

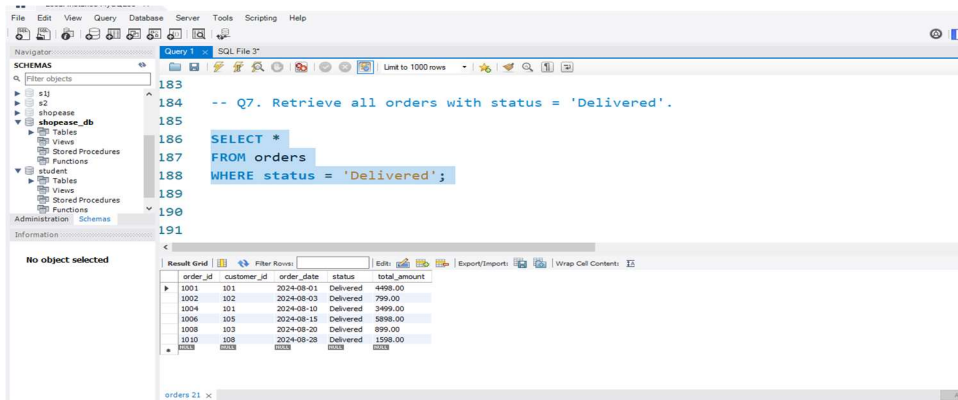
select * from products;
  
```

The result grid shows the following data for the products table:

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499.00	250
202	Cotton T-Shirt	Clothing	Levi's	799.00	499
203	Smart Watch	Electronics	Noise	2999.00	150
204	Running Shoes	Clothing	Nike	4599.00	120
205	Bluetooth Speaker	Electronics	2K	999.00	200
206	Bedsheet Set	Home	Spaces	1299.00	300
207	Laptop Stand	Electronics	AmazonBasics	899.00	180
208	Cushion Covers (Set)	Home	HomeCenter	599.00	399
209	Test Product	Electronics	TestBrand	-50.00	100

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.



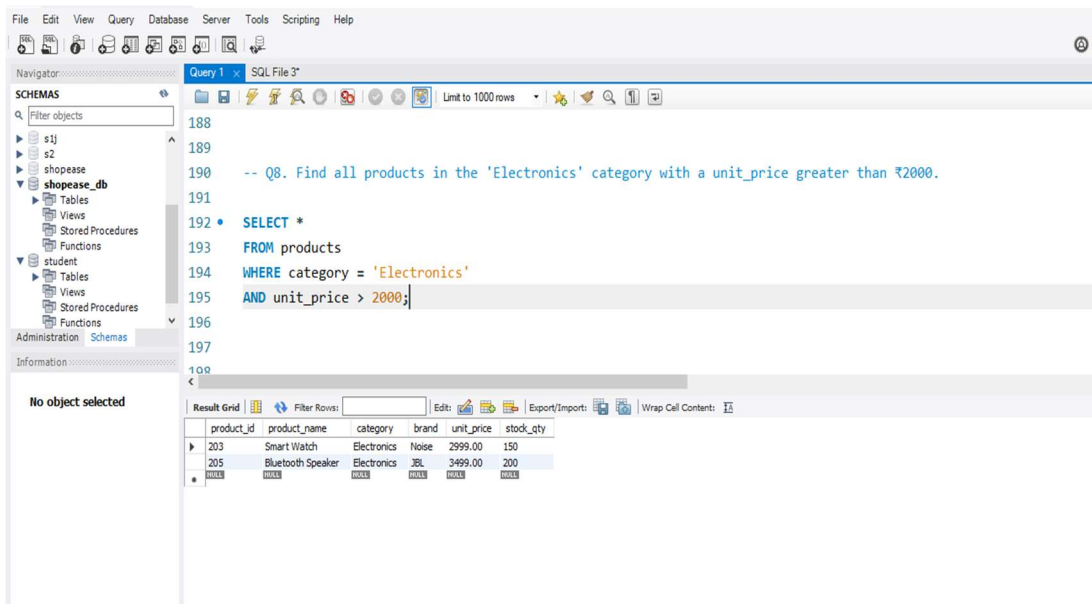
The screenshot shows a SQL query editor with the following SQL code:

```
-- Q7. Retrieve all orders with status = 'Delivered'.  
SELECT *  
FROM orders  
WHERE status = 'Delivered';
```

The result grid displays the following data:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.



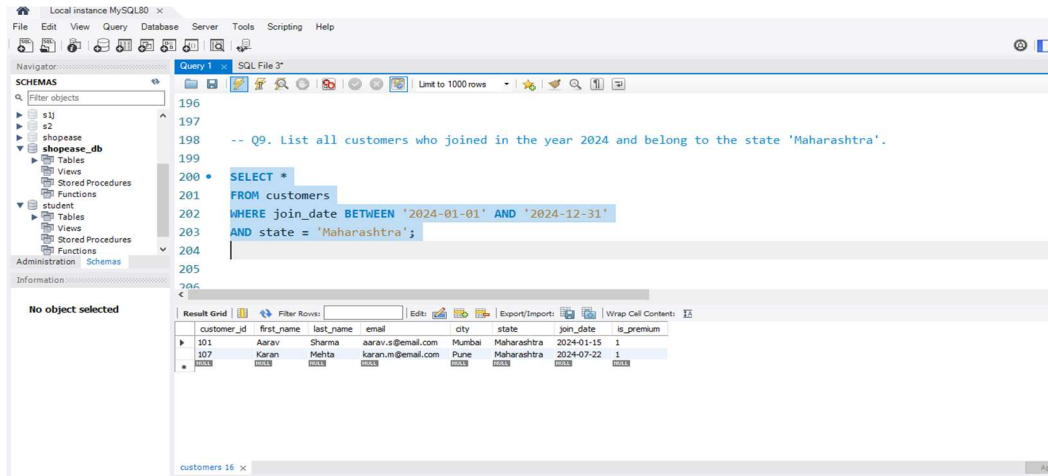
The screenshot shows a SQL query editor with the following SQL code:

```
-- Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.  
SELECT *  
FROM products  
WHERE category = 'Electronics'  
AND unit_price > 2000;
```

The result grid displays the following data:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

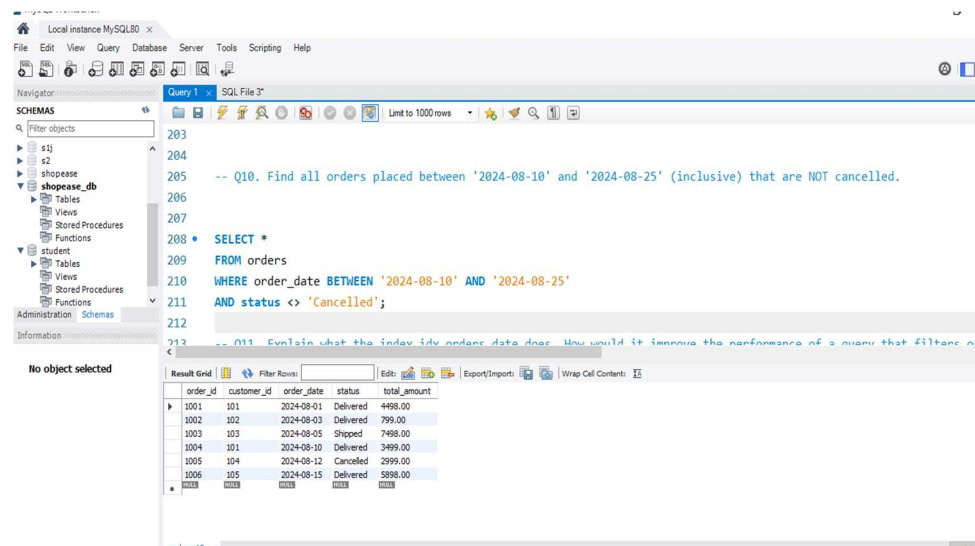


The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor shows a SQL query for Q9. The 'Result Grid' at the bottom displays the results of the query.

```
-- Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.  
  
SELECT *  
FROM customers  
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'  
AND state = 'Maharashtra';
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehhta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

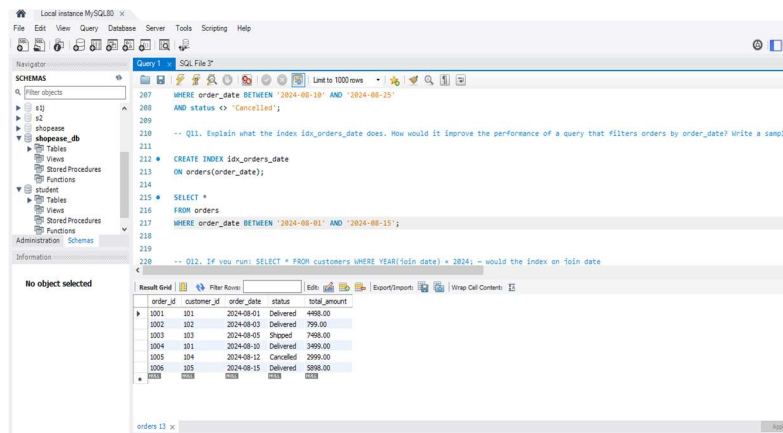


The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor shows a SQL query for Q10. The 'Result Grid' at the bottom displays the results of the query.

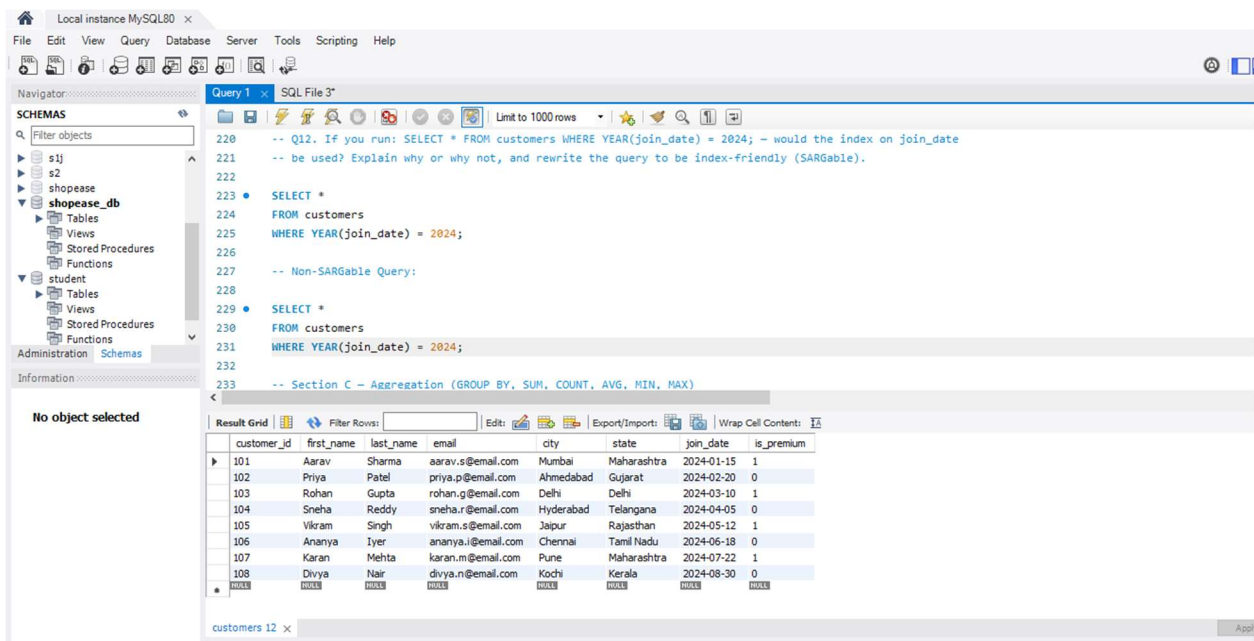
```
-- Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.  
  
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'  
AND status <> 'Cancelled';
```

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4488.00
1002	102	2024-08-03	Delivered	799.00
1003	103	2024-08-05	Shipped	7498.00
1004	101	2024-08-10	Delivered	3499.00
1005	104	2024-08-12	Cancelled	2999.00
1006	105	2024-08-15	Delivered	5898.00

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

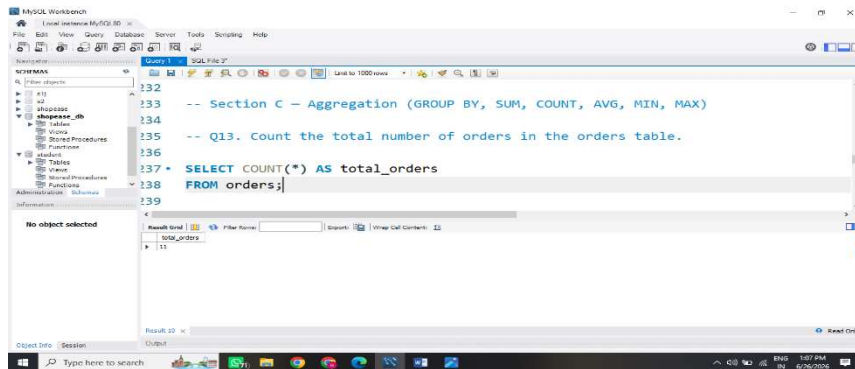


Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

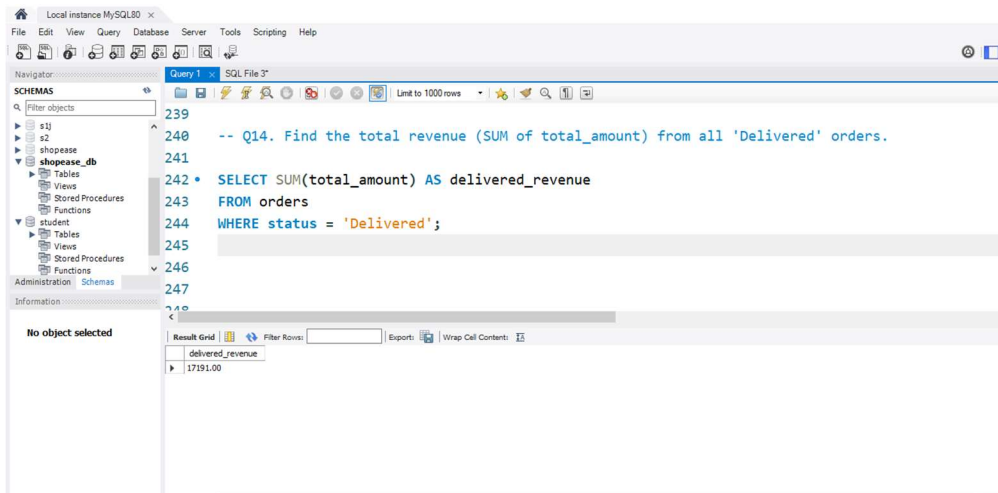


Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.



Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.



Q.15. Calculate the average unit_price of products in each category.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor contains a SQL query for Q15. The 'Result Grid' at the bottom shows the output of the query.

```

245
246
247
248 -- Q15. Calculate the average unit_price of products in each category.
249
250 * SELECT category, AVG(unit_price) AS avg_price
251 FROM products
252 GROUP BY category;
253
254
255 -- Q16. For each order status, find the count of orders and the total revenue. Sort the result

```

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	2	2897.00

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor contains a SQL query for Q16. The 'Result Grid' at the bottom shows the output of the query.

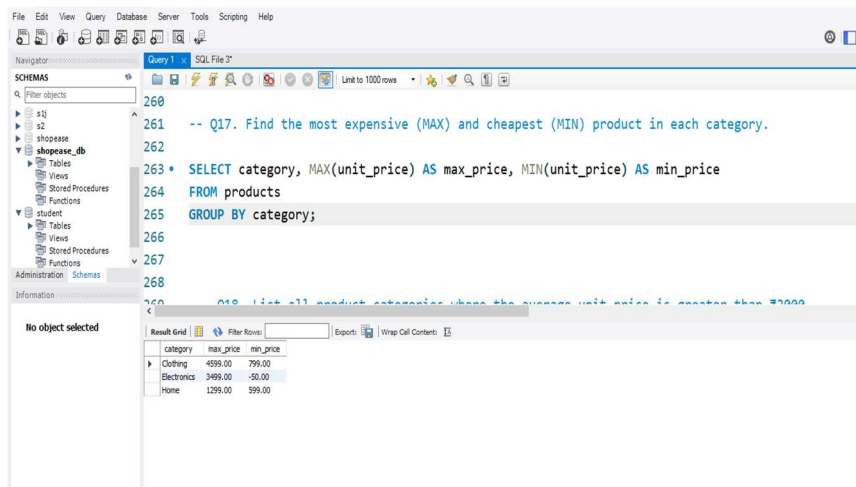
```

251
252 -- Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in de
253
254 * SELECT status, COUNT(*) AS order_count,
255 SUM(total_amount) AS total_revenue
256 FROM orders
257 GROUP BY status
258 ORDER BY total_revenue DESC;
259
260
261
262 -- Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	2	2897.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.



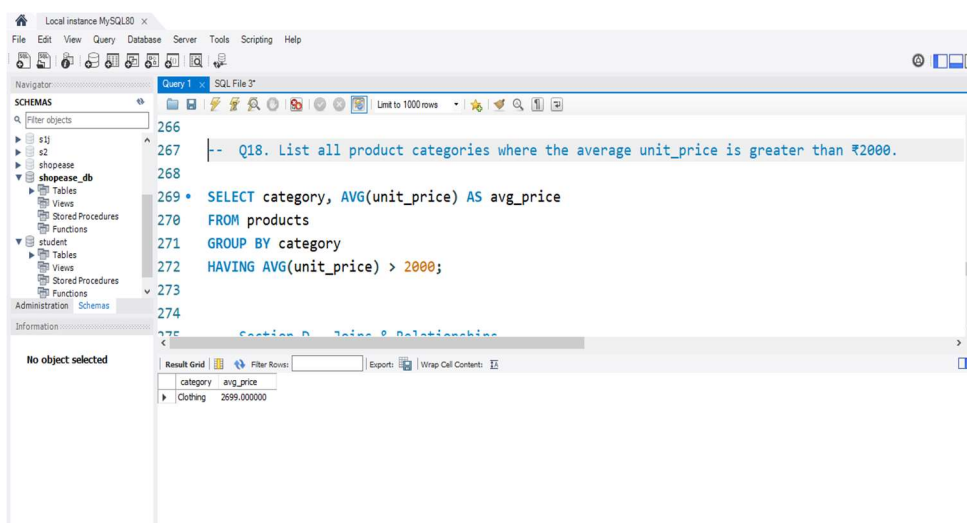
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopbase_db' selected. The main editor window shows a SQL query for Q17. The query is as follows:

```
-- Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.
SELECT category, MAX(unit_price) AS max_price, MIN(unit_price) AS min_price
FROM products
GROUP BY category;
```

The 'Result Grid' at the bottom displays the following data:

category	max_price	min_price
Clothing	4999.00	799.00
Electronics	3499.00	-50.00
Home	1299.00	999.00

Q18. List all product categories where the average unit_price is greater than ₹2000.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopbase_db' selected. The main editor window shows a SQL query for Q18. The query is as follows:

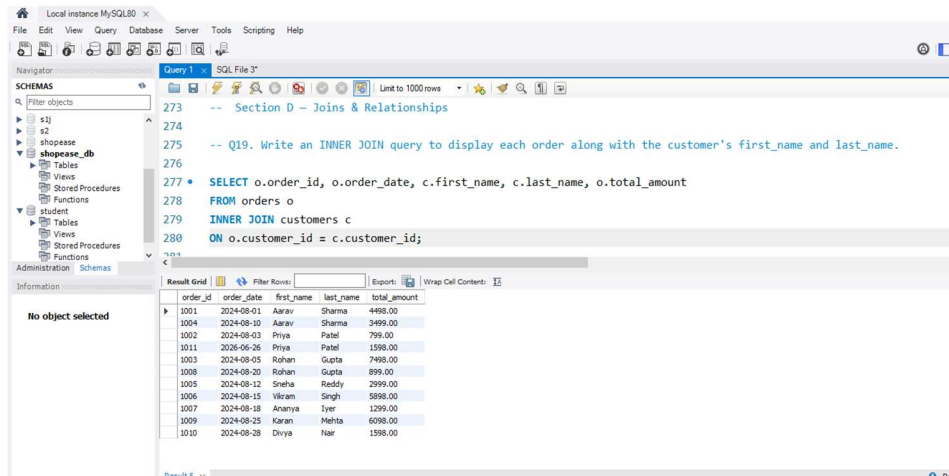
```
-- Q18. List all product categories where the average unit_price is greater than ₹2000.
SELECT category, AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

The 'Result Grid' at the bottom displays the following data:

category	avg_price
Clothing	2699.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.



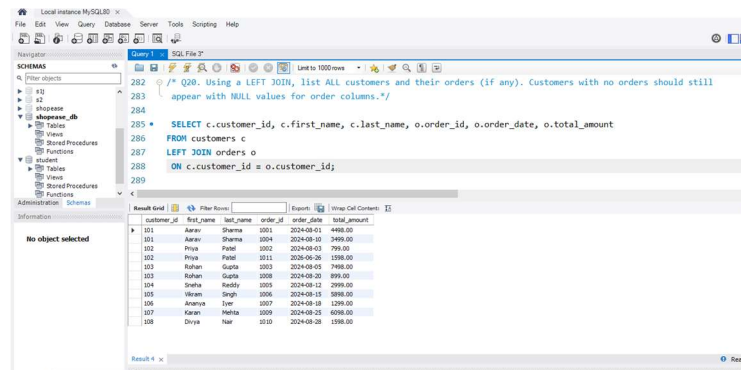
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopbase_db' selected. The main editor window shows a query titled 'Query 1' with the following SQL code:

```
-- Section D – Joins & Relationships
-- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.
SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

The 'Result Grid' at the bottom displays the results of the query, showing 10 rows of data with columns: order_id, order_date, first_name, last_name, and total_amount.

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2026-06-26	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Snaha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.



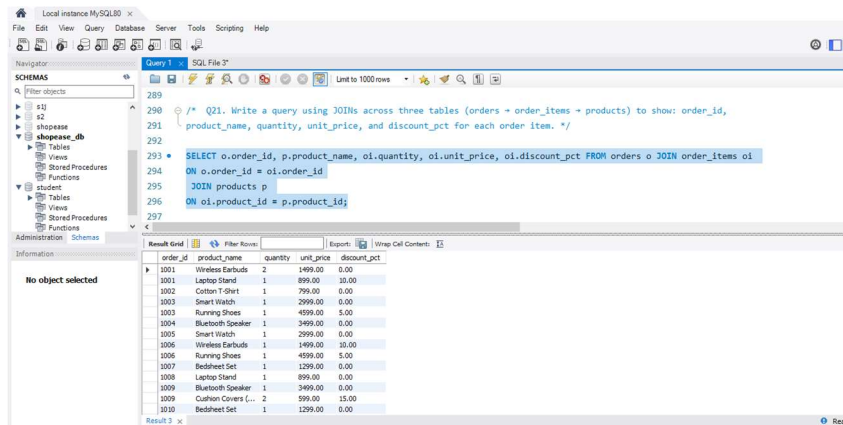
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopbase_db' selected. The main editor window shows a query titled 'Query 2' with the following SQL code:

```
/* Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.*/
SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

The 'Result Grid' at the bottom displays the results of the query, showing 10 rows of data with columns: customer_id, first_name, last_name, order_id, order_date, and total_amount. The results show all customers, with some having NULL values for order_id, order_date, and total_amount.

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
102	Priya	Patel	1011	2026-06-26	1598.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
105	Snaha	Reddy	1005	2024-08-12	2999.00
106	Vikram	Singh	1006	2024-08-15	5898.00
107	Ananya	Iyer	1007	2024-08-18	1299.00
109	Karan	Mehta	1009	2024-08-25	6098.00
110	Divya	Nair	1010	2024-08-28	1598.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.



Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN:

Returns all rows from the left table and matching rows from the right table.

```

SELECT * FROM customers c
LEFT JOIN
orders o ON c.customer_id = o.customer_id;

```

RIGHT JOIN:

Returns all rows from the right table and matching rows from the left table.

```

SELECT * FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;

```

FULL OUTER JOIN:

Returns all rows from both tables whether matches exist or not.

Use when you need:

- All customers
- All orders
- Including unmatched records from both tables

(MySQL does not directly support FULL OUTER JOIN.)

Q23. Identify all Foreign Key relationships in the schema.

Foreign Keys:

1. orders.customer_id → customers.customer_id

```
FOREIGN KEY(customer_id)
REFERENCES customers(customer_id)
```

2. order_items.order_id → orders.order_id

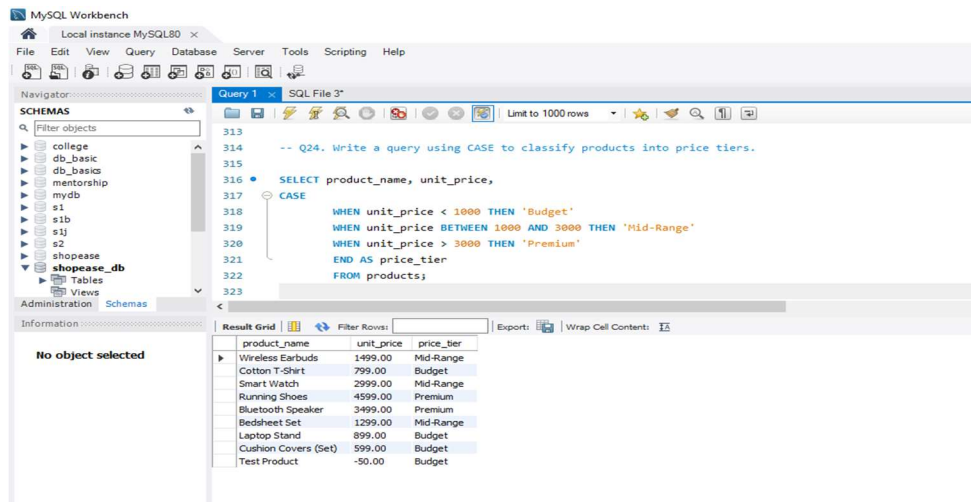
```
FOREIGN KEY(order_id)
REFERENCES orders(order_id)
```

3. order_items.product_id → products.product_id

```
FOREIGN KEY(product_id)
REFERENCES products(product_id)
```

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers.



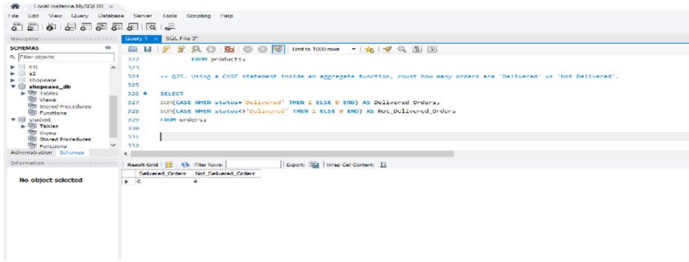
The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q24. Write a query using CASE to classify products into price tiers.
SELECT product_name, unit_price,
CASE
  WHEN unit_price < 1000 THEN 'Budget'
  WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
  WHEN unit_price > 3000 THEN 'Premium'
END AS price_tier
FROM products;
```

The Results window displays the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget
Test Product	-50.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered'.



Q26. Explain each letter of ACID.

A – Atomicity

Meaning: Either all steps of a transaction are completed successfully, or none of them are.

Example: If ₹5,000 is transferred from Account A to Account B and the system fails before the money reaches Account B, the entire transaction is cancelled (rolled back).

C – Consistency

Meaning: The database always remains correct and follows all rules before and after a transaction.

Example: An order cannot be placed for a customer who does not exist in the `customers` table.

I – Isolation

Meaning: Multiple transactions running at the same time do not affect each other.

Example: If two customers buy the last product simultaneously, the database processes the transactions separately to avoid incorrect stock updates.

D – Durability

Meaning: Once a transaction is committed, the changes are saved permanently.

Example: After a successful online payment, the order remains saved even if the system crashes.

Importance: Ensures that committed data is never lost.

Q27. Write a SQL transaction that does the following atomically:

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- s1
- s2
- shopease
- shopease_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- student
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Query 1 x SQL File 3*

Limit to 1000 rows

```

334 -- insert new order
335 INSERT INTO orders
336 (order_id, customer_id, order_date, status, total_amount)
337 VALUES
338 (1001, 100, CURRENT_TIMESTAMP(), 'Pending', 1100.00);
339
340 -- insert order items
341 INSERT INTO order_items
342 (item_id, order_id, product_id, quantity, unit_price, discount_pct)
343 VALUES
344 (1001, 1001, 202, 1, 100.00, 0);
345 (1001, 1001, 200, 1, 100.00, 0);
346
347 -- update product stock
348 UPDATE products
349 SET stock_qty = stock_qty - 1
350 WHERE product_id = 202;
351
352 UPDATE products
353 SET stock_qty = stock_qty - 1
354 WHERE product_id = 200;
355
356 COMMIT;
357
358

```

Output

Action Output

#	Time	Action	Message	Duration
6	12:31:24	INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct) ...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 s
7	12:31:24	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 202	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 s
8	12:31:24	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 200	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 s
9	12:31:24	COMMIT	0 row(s) affected	0.000 s
10	12:32:12	ROLLBACK	0 row(s) affected	0.000 s

Q28. Display the Top 3 Most Expensive Products

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- s1
- s2
- shopease
- shopease_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- student
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Query 1 x SQL File 3*

Limit to 1000 rows

```

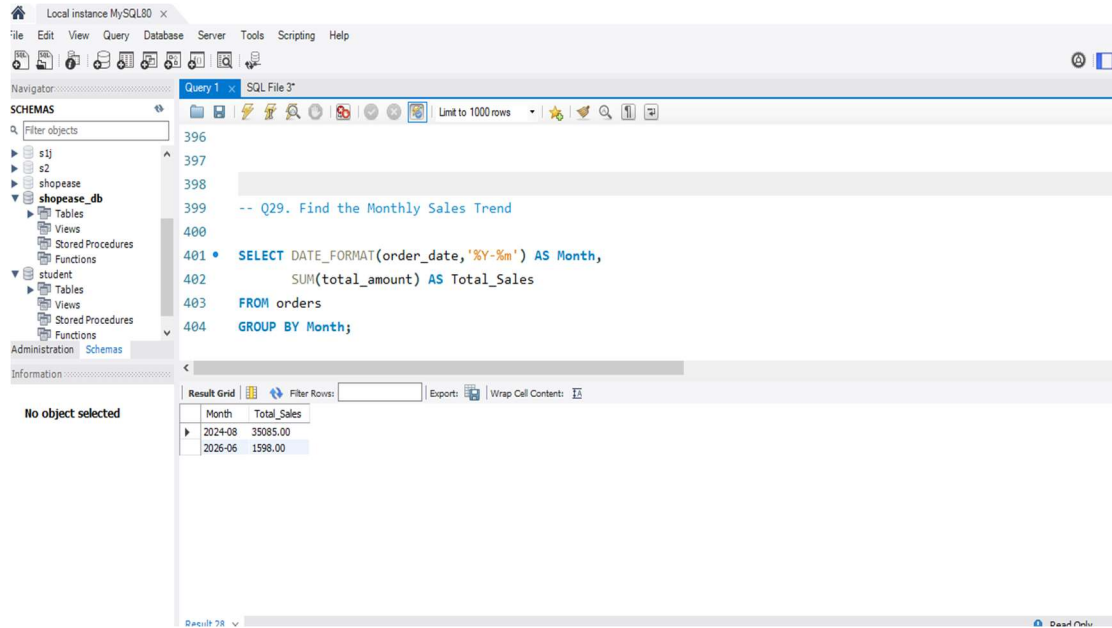
387
388 -- Q28. Display the Top 3 Most Expensive Products
389
390 SELECT product_name,
391        category,
392        unit_price
393 FROM products
394 ORDER BY unit_price DESC
395 LIMIT 3;
396

```

Result Grid

product_name	category	unit_price
Running Shoes	Clothing	4599.00
Bluetooth Speaker	Electronics	3499.00
Smart Watch	Electronics	2999.00

Q29. Find the Monthly Sales Trend



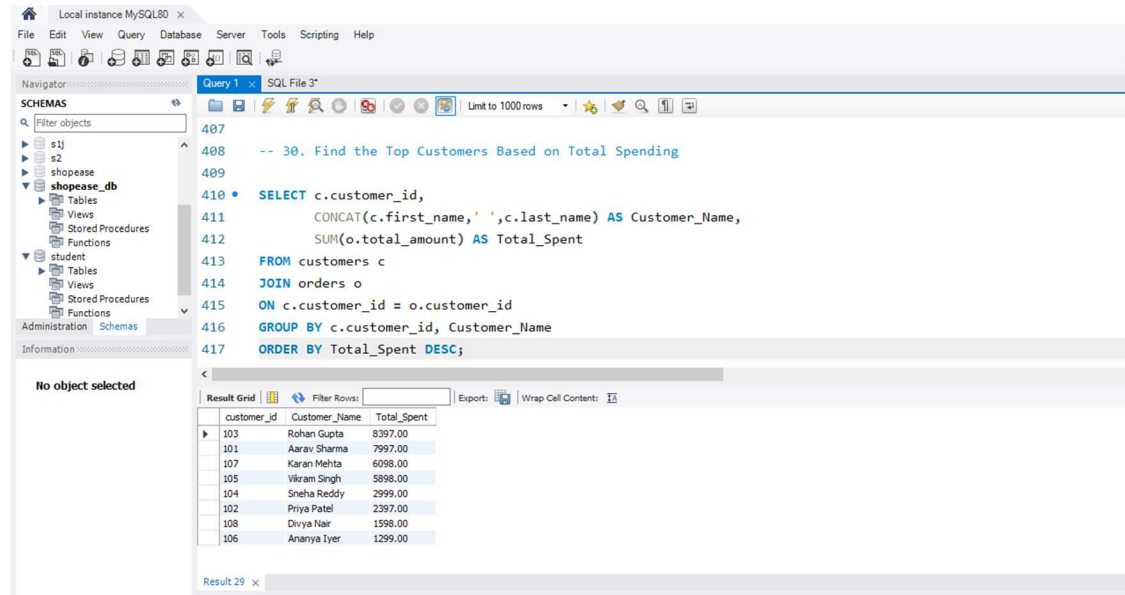
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor shows a SQL query for finding the monthly sales trend. The query is as follows:

```
-- Q29. Find the Monthly Sales Trend
SELECT DATE_FORMAT(order_date, '%Y-%m') AS Month,
       SUM(total_amount) AS Total_Sales
FROM orders
GROUP BY Month;
```

The result grid shows the following data:

Month	Total_Sales
2024-08	35085.00
2026-06	1598.00

Q.30. Find the Top Customers Based on Total Spending



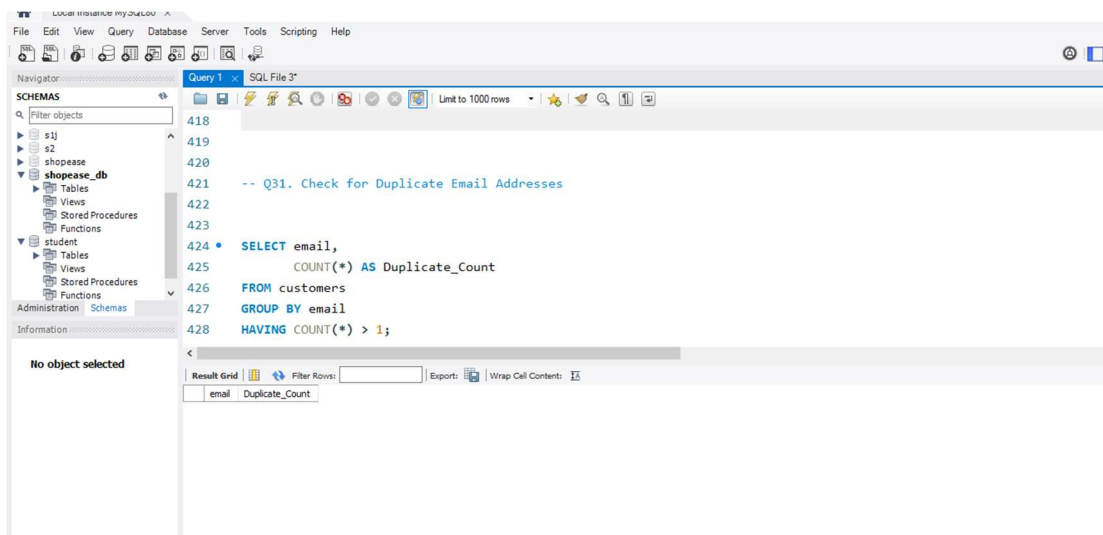
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'shopease_db' selected. The main editor shows a SQL query for finding the top customers based on total spending. The query is as follows:

```
-- 30. Find the Top Customers Based on Total Spending
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS Customer_Name,
       SUM(o.total_amount) AS Total_Spent
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, Customer_Name
ORDER BY Total_Spent DESC;
```

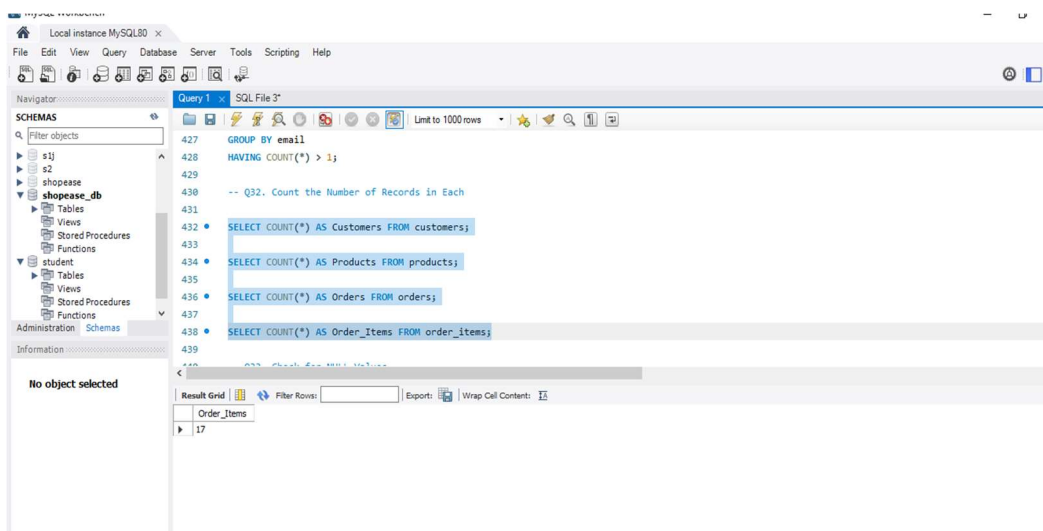
The result grid shows the following data:

customer_id	Customer_Name	Total_Spent
103	Rohan Gupta	8397.00
101	Aarav Sharma	7997.00
107	Karan Mehta	6098.00
105	Vikram Singh	5898.00
104	Sneha Reddy	2999.00
102	Priya Patel	2397.00
108	Divya Nair	1598.00
106	Ananya Iyer	1299.00

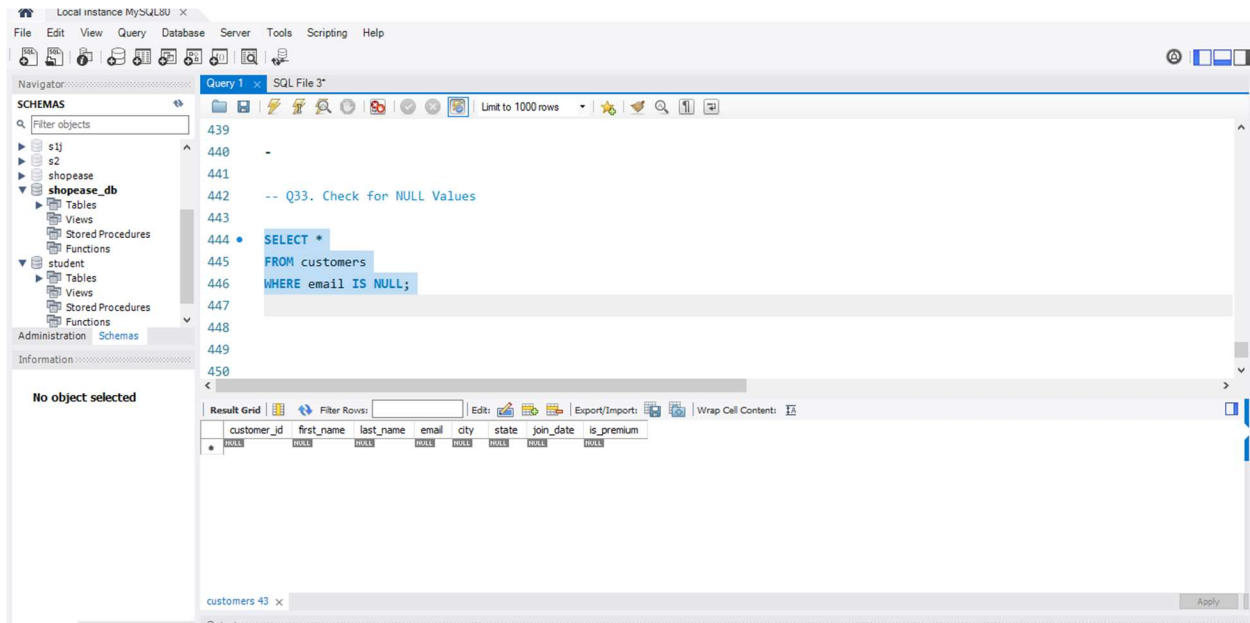
Q31. Check for Duplicate Email Addresses



Q32. Count the Number of Records in Each



Q33. Check for NULL Values



Brief Insights:

- The database consists of customer, product, order, and order item information stored in related tables.
- Delivered orders contribute the highest revenue among all order statuses.
- Electronics and Clothing products have the highest average selling prices.
- No duplicate email addresses were found due to the `UNIQUE` constraint.
- Aggregate functions such as `SUM()`, `COUNT()`, and `AVG()` provide valuable business summaries.
- JOIN operations combine customer, order, and product information effectively.
- Primary Keys and Foreign Keys maintain referential integrity.
- SQL queries help transform raw sales data into meaningful business insights for decision-making.

Conclusion:

This assignment successfully demonstrates the use of SQL for analyzing sales data using MySQL Workbench. The database was created, populated with sample data, and analyzed using filtering, aggregation, sorting, joins, CASE statements, and transactions. Business reports such as top customers, monthly sales trends, and product analysis were generated successfully. Data validation queries confirmed data integrity, while constraints and indexes ensured reliable and efficient database operations. Overall, the assignment shows how SQL can be effectively used for real-world business data analysis and decision-making.